Lab 5: Static Code Analysis

Software Engineering Laboratory — Submission Report

Student Name: Kruthik Reddy S
SRN: PES2UG24CS241
Date: 14 October 2026

1. Objective

To improve Python code quality, security, and style by applying Pylint, Bandit, and Flake8; documenting issues found; applying corrective changes; and rerunning the tools to verify the changes.

2. Repository and Tools

Assigned repository: https://github.com/SETAPESU26/L5_50_flight_reservation_system
Analyzed file: flight_reservation_system.py
Environment: Python 3.10.12 virtual environment (venv)
Tools: Pylint, Bandit, and Flake8

3. Issues Identified and Fixes Applied

Original finding	Tool / severity	Correction applied
Mutable default argument: baggage_tags=[]	Pylint / correctness	Changed to None and initialized a new list inside the function.
Bare except: in cancellation	Pylint, Flake8, Bandit / low	Removed broad exception suppression and explicitly validated the flight number.
eval(surcharge_expr)	Bandit B307 / medium	Removed eval; surcharge is accepted as a numeric parameter and validated.
MD5 used for passenger digest	Bandit B324 / high	Replaced MD5 with SHA-256. This digest is not a substitute for a password-hashing system.
random.randint used for PIN	Bandit B311 / low	Used secrets.randbelow for cryptographically stronger PIN generation.
os.system invoked with filename	Bandit B605 / high	Removed shell execution and wrote the manifest using a file context manager.
Hardcoded API-like secret and printed token	Bandit B105 / low	Removed the unused hardcoded secret and its output.
Unused import, unused variables, long line and naming issues	Pylint / Flake8	Removed unused imports/variables, used clearer names, split long expressions and added documentation.
File handling without explicit encoding	Pylint	Opened the manifest using encoding='utf-8' and a context manager.
Missing module docstring / final newline	Pylint / Flake8	Added a module docstring and ensured a final newline.
Module ran main() unconditionally	Maintainability	Used the if __name__ == '__main__' guard.

4. Before-and-After Results

Tool	Initial result	Final verified result
Pylint	4.00/10	10.00/10
Bandit	6 findings: 2 high, 1 medium, 3 low	No issues identified
Flake8	9 reported violations	No output; report file was 0 bytes
Program execution	Original program inspected	Corrected program executed successfully

Observed execution output included a baggage fee of 517.5, a SHA-256 passenger digest, a generated four-digit PIN, a successful manifest export, and an available-flight count of 3.

5. Reflection Questions

Q1. What is static code analysis?
Static code analysis examines source code without running the program. It can identify potential defects, security risks, and style or maintainability problems early in development.

Q2. How did Pylint, Bandit, and Flake8 differ?
Pylint assessed broader code quality, naming, documentation, and suspicious patterns. Bandit focused on security-sensitive patterns such as eval, weak hashing, shell execution, and hardcoded secrets. Flake8 reported style and formatting violations, including long lines, unused imports, and bare except.

Q3. Which issues were most important to fix and why?
The unsafe eval call and shell invocation were important because they could enable unintended code or command execution. The weak hash and non-cryptographic random number generator were also security concerns. The hardcoded secret was removed to avoid exposing a credential-like value in source code or output.

Q4. What did rerunning the tools demonstrate?
The final runs reported a Pylint score of 10.00/10, no Bandit findings, and no Flake8 output. This shows that the reported static-analysis findings were addressed for the submitted file. It does not prove that all possible functional defects are absent; functional tests are still necessary.

Q5. How does static analysis help software development?
It provides repeatable feedback before execution or deployment, helps developers prioritize defects, supports consistent coding practices, and makes security and maintainability issues easier to detect during development.

6. Verification and Limitations

The corrected script was run successfully in the terminal. The final observed reports showed Pylint 10.00/10, Bandit reporting no issues, and an empty Flake8 report. A separate functional limitation remains worth testing: the cancellation operation should only restore a seat when a valid booking exists. Static analysis results alone do not validate all business logic. The generated report files and the corrected Python source should be included with the submission if required by the instructor.

7. Files to Submit

1. flight_reservation_system.py (corrected source)
2. pylint_report.txt
3. bandit_report.txt
4. flake8_report.txt
5. This PDF report

Include the original backup only if requested by the instructor. Do not submit the venv directory unless specifically required.